

ETF Risk-Signal Diversification

OPRE-Oriented Journal Article Blueprint and Implementation Report

Dynamic Risk Control in Bayesian-CVaR ETF Allocation

Prepared from the uploaded code archive, Risk-Signal Diversification document, and Vega presentation draft

Date: 2026-07-12

Purpose. This PDF is not the final journal manuscript. It is a step-by-step writing blueprint and implementation report: it explains the current code pipeline, maps code to paper sections, defines the mathematical narrative, separates known ideas from defensible novelty, and lists the improvements needed before submitting to Operations Research.

Contents

1. 1. Executive summary and article thesis
2. 2. Positioning for Operations Research
3. 3. Current code state and deliverable summary
4. 4. ETF universe construction from `monthly_etf_filtration.py` and `monthly_etf_filtration_2.py`
5. 5. Market and quantitative rationale for the universe metrics
6. 6. Walk-forward portfolio pipeline in `scripts/runs`
7. 7. Bayesian posterior-predictive scenario engine
8. 8. CVaR-constrained allocation and execution
9. 9. Risk-signal diversification overlays
10. 10. New smart-signal and sizing layer
11. 11. Train/test validation and extended metrics
12. 12. Plotly HTML dashboard design
13. 13. Known ideas versus defensible novelty
14. 14. Mathematical propositions and proof sketches
15. 15. Empirical design, hypotheses, and inference
16. 16. How to write the OPRE manuscript section by section
17. 17. Required upgrades before submission
18. 18. Code usage and file map
19. 19. References and reproducibility checklist

1. Executive summary and article thesis

The paper should be framed as a dynamic stochastic-optimization and risk-control problem, not as a simple trading-strategy backtest. At each monthly rebalance date, the decision maker observes only past ETF prices, volumes, realized returns, the current eligible universe, and the previous portfolio. The decision is to choose long-only weights and a total exposure level that balance expected return, scenario-based tail risk, transaction costs, and changing market conditions.

The strongest thesis is: a forward model-implied tail-risk signal, computed from the same posterior-predictive scenario engine that defines the CVaR allocation constraint, contains information that is different from a backward realized-volatility signal. Because the two exposure timers are only weakly correlated, their multiplicative combination functions as risk-signal diversification. The claim does not need to be that the strategy always maximizes terminal wealth in a bull market. The safer and stronger claim is risk transformation: lower volatility, lower drawdown, lower realized tail loss, and higher downside-adjusted performance.

Article component	Recommended framing
Decision problem	Monthly portfolio and exposure choice with changing eligible ETF universe, noisy inputs, and transaction costs.
Core method	Scenario-based stochastic optimization with a coherent CVaR constraint and adaptive exposure control.
Main mechanism	Conditioning total exposure on two different risk signals: backward realized volatility and forward model-implied CVaR.
Main empirical claim	Downside protection is robust; security-selection alpha is not the central claim.
OPRE angle	A reproducible decision policy with mathematical structure, algorithms, validation, and stress-tested empirical evidence.

1.1 Suggested title and one-paragraph claim

A clean finance title is: Risk-Signal Diversification: Combining Forward Posterior-Predictive CVaR with Realized-Volatility Timing in Long-Only ETF Allocation.

A stronger Operations Research title is: A Model-Consistent Tail-Risk Timing Policy for Dynamic Long-Only Portfolio Optimization with Transaction Costs.

Article claim. We design a monthly walk-forward policy for a long-only ETF book. Each month, a causal investable universe is built from liquidity, momentum, benchmark-relative, and risk metrics. A Bayesian shrinkage scenario engine generates next-period return scenarios. The allocation solves a maximum-predictive-return problem subject to a scenario-based CVaR budget, long-only weight caps, and turnover costs. The same scenario fan also produces a forward CVaR forecast of the held book. Exposure is then scaled by two risk timers: backward realized volatility and forward model-CVaR. The empirical result is not that the optimizer discovers persistent security-selection alpha; instead, the value comes from conditioning exposure on two distinct risk signals.

2. Positioning for Operations Research

Operations Research is a suitable target only if the paper is written around the decision system and its mathematical structure. The official editorial statement emphasizes analytical methods for improved decision-making, methodological and empirical rigor, and contributions that can be demonstrated with mathematics, data, software, or computation. Therefore, the manuscript should foreground causal filtrations, stochastic scenario generation, coherent risk constraints, bounded adaptive policies, and validation against overfitting.

The practical page constraint matters. A regular manuscript is normally capped at about 30 pages excluding references, and the empirical details, proofs, extra plots, code map, and additional robustness checks should be placed in an electronic companion or code appendix.

OPRE expectation	How to satisfy it in this project
------------------	-----------------------------------

Clear decision problem	Define the state S_t , eligible action set A_t , prior portfolio w_{t-1} , stochastic scenarios R_t , and action (w_t, k_t) .
Analytical contribution	State propositions: causality of universe construction, CVaR LP equivalence, tail-risk budget guarantee, turnover bound, bounded dynamic controller, exposure homogeneity, and product-timer complementarity.
Algorithmic clarity	Provide pseudocode for monthly universe screening, scenario generation, optimization, execution, overlays, and train/test parameter selection.
Empirical rigor	Use walk-forward tests, 1/N and Markowitz benchmarks, PSR/DSR, paired bootstrap, transaction costs, and ablation studies.
Reproducibility	Submit code, README, config files, generated artifacts, and a list of all model variants used in the deflated Sharpe calculation.

3. Current code state and deliverable summary

The uploaded repository was inspected and extended while preserving the original scripts structure. The full historical backtest was not executed because the raw ETF datasets required by the pipeline are not included in the sandbox. Instead, the repository was syntax-checked with compileall, and the Plotly dashboard generator was smoke-tested on synthetic monthly ETF data. This is important: the code package is ready for your environment, but final numerical claims must be rerun on your real data before being used in a journal submission.

Area	Implemented or preserved in code package
Notebook-to-console migration	FINAL_results-style logic is moved toward console entry points such as scripts/runs/final_results_simple.py and shared post-processing modules.
Smart signals	scripts/risk_controls/smart_signals.py computes ER, ATR regime, liquidity, breakout strength, normalized spike, pullback ratio, net move, GARCH-style volatility, scenario Kelly, and risk-reward signals.
Train/test reporting	scripts/results/stats.py and scripts/results/interactive_report.py include train/test metrics, win rate, average drawdown, expectancy, profit factor, rolling Sharpe, and rolling Sortino.
Dashboard	scripts/results/interactive_report.py generates a self-contained Plotly HTML report with performance, drawdown, train/test metrics, rolling ratios, histograms, turnover, heatmaps, risk matrices, and monthly tables.
Risk-matrix export	scripts/runs/common_postprocess.py, scripts/runs/run.py, scripts/runs/strategy_run.py, and scripts/models/bayessian.py were extended so the dashboard can display the actual point-in-time covariance/correlation matrix used by the model.
Run integration	scripts/runs/run.py and scripts/runs/strategy_run.py call shared post-processing, save smart-signal audits, risk forecasts, dashboard outputs, and risk matrix artifacts where data is available.

Main limitation. The delivered numerical sample dashboard is a smoke test; it is not a research result. It proves that the report generator works with the expected artifact schema, not that the strategy has a certain Sharpe ratio on real data.

4. ETF universe construction from monthly_etf_filtration.py and monthly_etf_filtration_2.py

The ETF universe procedure is one of the most important parts of the paper because it defines what was investable at each rebalance date. It must be described as a causal, point-in-time selection policy. The code uses price and volume matrices, aligns common ETF columns, constructs monthly windows, computes cross-sectional

metrics for each ETF, ranks ETFs by an aggregate score, applies basic quality filters, optionally applies a benchmark-relative filter against SPY, and exports the top-N selected ETFs for each month.

4.1 Active metrics computed each month

Metric	Code meaning	Economic meaning
price_change	End price divided by start price minus one over the monthly or rolling window.	Captures recent momentum and immediate investor demand.
excess_return_vs_benchmark	ETF price_change minus SPY price_change.	Separates ETF strength from general market beta.
risk_adjusted_return	Price change divided by realized volatility.	Prefers cleaner returns, not only high raw returns.
risk_adjusted_excess_return	Benchmark-relative return divided by ETF volatility.	Combines relative strength and risk normalization.
positive_days_ratio	Fraction of days with positive return.	Measures persistence instead of one-day jumps.
total_volume and dollar_volume	Share volume and price times volume.	Tradability and approximate implementation capacity.
volume_change	Current total volume relative to the previous month.	Detects liquidity improvement or deterioration.
momentum_ratio_vs_benchmark	Ratio of ETF gross return to benchmark gross return minus one.	Alternative relative-strength statistic robust to scale.
volatility and downside_volatility	Standard deviation of daily returns and negative-return volatility.	Risk and downside asymmetry.
drawdown_abs	Absolute value of max drawdown in the window.	Penalty for path-dependent loss.
amihud_illiquidity	Mean absolute return divided by dollar volume.	Penalty for price impact and poor liquidity.
beta_to_benchmark and corr_to_benchmark	Regression beta and correlation to SPY.	Used for diagnostics and possible future risk controls.

4.2 Aggregate score

The code converts heterogeneous metrics into percentile ranks and combines them with explicit weights. Higher-is-better metrics include price_change, excess return, risk-adjusted return, risk-adjusted excess return, positive days ratio, log total volume, log dollar volume, volume change, and momentum ratio versus benchmark. Lower-is-better penalties include volatility, downside volatility, absolute drawdown, and Amihud illiquidity. This rank-based design is useful because each metric has a different unit and scale.

```
score_i,t = (sum_j a_j * rank_high(metric_j,i,t)
            + sum_k b_k * rank_low(metric_k,i,t)) / (sum_j a_j + sum_k b_k)
```

```
rank_high(x) = percentile rank where large values are good
rank_low(x)  = 1 - percentile rank where small values are good
```

4.3 Difference between the two filtration scripts

Script	Main selection idea	Recommended article use
monthly_etf_filtration_2.py	Uses rolling windows and an EMA-smoothed aggregate score: $score_t = \alpha * current_score_t + (1-\alpha) * previous_score_t$.	Treat as the newer, turnover-aware candidate universe policy because EMA smoothing can reduce month-to-month instability.
monthly_etf_filtration.py	Uses the same metric family but computes a rolling_aggregate_score by averaging each ETF score over a configurable number of historical months.	Treat as a robustness variant. It is useful for proving that conclusions do not depend on one smoothing method.

4.4 What is novel and what is not

The individual metrics are not new. Momentum, liquidity, volatility, drawdown, and benchmark-relative strength are standard market features. The defensible contribution is the point-in-time integration: a changing ETF universe is formed causally, with liquidity and history safeguards, and then passed into a stochastic optimization

and risk-timing loop. To make this publishable, compare it to simpler screens: top dollar volume only, top market cap or AUM if available, fixed ETF list, no benchmark filter, and random top-N from liquid ETFs.

5. Market and quantitative rationale for the universe metrics

The universe filter should be written as an operational risk-control layer rather than an alpha model. The purpose is to prevent the optimizer from allocating to illiquid, unstable, newly listed, or pathologically noisy ETFs. This matters because optimizers magnify input errors. A small error in expected return or covariance can become a large portfolio concentration if the feasible universe contains unstable instruments.

Metric family	Why it is useful	How it can fail	Improvement before publication
Momentum and benchmark-relative return	Finds ETFs with persistent demand and relative strength.	Can chase short-term reversal or sector crowding.	Add 12-1 momentum and compare with 1-month momentum. Test no-momentum universe.
Risk-adjusted return	Prevents high return from dominating if it comes with extreme volatility.	May overfit if volatility is estimated on too short a window.	Test 21, 63, 126-day windows and EW volatility.
Liquidity and dollar volume	Controls tradability and implementation cost.	Volume spikes can be temporary and misleading.	Use rolling median dollar volume and bid-ask spread if available.
Volume change	Detects newly tradable funds and liquidity deterioration.	Can admit event-driven spikes.	Winsorize volume change and require minimum rolling dollar volume.
Drawdown and downside volatility	Screens path-dependent tail risk.	Can exclude assets after temporary selloffs that later rebound.	Treat as penalty, not hard exclusion; add stress-period sensitivity.
Amihud illiquidity	Approximate price-impact penalty.	Crude when close prices and daily volume are the only inputs.	Use quoted spreads or high-low spread estimators if available.

6. Walk-forward portfolio pipeline in scripts/runs

The runners should be described as a monthly walk-forward loop. A clean algorithm for the paper is below.

Algorithm: Monthly ETF risk-signal allocation

Input: prices, volumes, rebalance dates, previous weights w_{t-1}

For each rebalance month t :

1. Use only data dated before or at t .
2. Build eligible ETF universe A_t with filtration script.
3. Estimate predictive means, covariance, and scenario fan R_t .
4. Solve CVaR-constrained allocation for target weights w_t^{target} .
5. Optionally adjust weights with smart signal and bounded sizing layer.
6. Partially rebalance: $w_t^{\text{exec}} = w_{t-1} + a_t (w_t^{\text{target}} - w_{t-1})$.
7. Compute transaction cost from turnover.
8. Compute net realized return until next rebalance.
9. Compute forward CVaR risk signal from the same scenarios.
10. Scale exposure using backward vol timer, forward CVaR timer, or their product.
11. Save PnL, weights, forecasts, risk matrices, and dashboard artifacts.

In the paper, every object should be indexed by the information set F_t . This makes it clear that the universe, model inputs, optimizer parameters, and exposure overlay are causal. Avoid describing anything as if full-sample future information was known at the rebalance date.

7. Bayesian posterior-predictive scenario engine

The scenario engine is the bridge between portfolio optimization and forward risk timing. It estimates predictive mean and covariance on a rolling historical window and generates S simulated next-month outcomes for the

eligible ETF set. The same scenario fan is used twice: first to define the CVaR constraint in the optimizer, and second to read the held portfolio CVaR as a forward risk signal.

```
Let A_t be the eligible ETF universe and N_t = |A_t|.
Estimate mu_t and Sigma_t from historical returns up to time t.
Draw Z_s ~ N(0, I), s = 1, ..., S.
Generate scenario returns: R_{s,t} = mu_t + L_t Z_s, where L_t L_t' = Sigma_t.
For weights w, scenario portfolio return is y_s(w) = R_{s,t}' w.
Loss is l_s(w) = -y_s(w).
```

The code stores the model covariance and correlation matrices used at each rebalance so the HTML dashboard can show the actual risk matrix rather than an ex-post matrix computed only for reporting. This supports auditability and is important for a journal code appendix.

8. CVaR-constrained allocation and execution

CVaR is used carefully. The safer article story is not "we maximize CVaR" or "CVaR alone creates alpha." Instead, CVaR is a coherent downside risk budget. The optimizer searches for expected return subject to a cap on predicted tail loss. Turnover penalties and partial rebalancing make the decision closer to actual implementation.

```
Empirical CVaR at level alpha:
CVaR_alpha(w) = min_eta [ eta + (1 / ((1-alpha)S)) * sum_s max(l_s(w) - eta, 0) ]
```

```
Portfolio problem:
maximize    mu_t' w - lambda_t ||w - w_{t-1}||_1
subject to  CVaR_alpha_t(w) <= c_t
            sum_i w_i = 1
            lower_i <= w_i <= upper_i
            w_i >= 0
```

A useful transparent budget is $c_t = \kappa * \text{CVaR_alpha_t}(w_{\text{equal}})$. If $\kappa \leq 1$, the optimizer is allowed to improve expected return only while staying at or below the predicted tail risk of an equal-weight book. This converts the optimizer from a pure return chaser into a risk-budgeted decision rule.

Execution is partial: $w_{\text{exec}} = w_{\text{prev}} + a_t (w_{\text{target}} - w_{\text{prev}})$, with a_t in $[0,1]$. This strictly bounds turnover relative to a full rebalance and gives a simple proposition for the paper.

9. Risk-signal diversification overlays

The core idea in the Word document and presentation is to combine a backward risk signal and a forward risk signal. Backward realized volatility is based on trailing portfolio volatility and is easy to compute, but it is naturally reactive. Forward model-CVaR is read from the posterior-predictive scenario fan and can warn about scenario-implied tail risk even when realized volatility has not yet fully reacted. The product overlay de-risks when either signal flags.

Timer	Data source	Interpretation	Risk
Backward realized-vol timer	Trailing realized volatility of the reconstructed book.	Variance risk has recently increased; scale exposure toward target volatility.	Can be late after sudden regime shifts and can overreact after the shock.
Forward model-CVaR timer	Posterior-predictive scenarios from the allocation engine.	Model-implied tail loss is high relative to causal target.	Can be noisy if scenario engine or covariance estimate is unstable.
Combined product timer	$k_t = k_{\text{vol},t} * k_{\text{cvar},t}$ or matched-exposure variant.	De-risk when either timer warns; broader coverage of danger states.	Can sacrifice upside in bull markets and requires careful matched-exposure comparison.

The paper should emphasize complementarity, not only performance. Show the correlation between exposure paths, the share of months each timer flags alone, and stress-case studies where one timer reacts earlier than the other. This turns the contribution from "another overlay" into risk-signal diversification.

10. New smart-signal and sizing layer

The uploaded code package now includes a richer smart-signal layer. These signals should not replace the optimizer. They should act as ranking, gating, or bounded sizing controls. This separation is important: alpha-like signals and risk controls should not be mixed so aggressively that the paper loses a clear decision structure.

Signal	Formula idea	Role in portfolio
Efficiency ratio	$\text{abs}(\text{net price move}) / \text{sum}(\text{abs}(\text{path moves}))$.	Rewards clean trends; penalizes noisy paths.
ATR volatility regime	short-window ATR proxy / long-window ATR proxy.	Detects elevated current volatility; used to shrink risk.
Liquidity score	recent dollar volume or volume proxy relative to cross-section.	Rewards tradability and capacity.
Breakout strength	location of price inside recent high-low channel.	Measures trend extension or breakout pressure.
Normalized spike	latest return divided by rolling volatility.	Penalizes unstable one-day jumps and exhaustion moves.
Pullback ratio	distance from recent high divided by recent range.	Rewards moderate pullbacks, avoids collapse.
Net move	rolling cumulative return.	Keeps direction information separate from path efficiency.
GARCH-style volatility	$\omega + \alpha * \text{shock}^2 + \beta * \text{variance recursion}$.	Volatility-clustering forecast for risk scaling or matrix diagonal rescaling.
Kelly fraction	estimated edge divided by risk, capped and floored.	Bounded conviction scaler, not final raw weight.
Risk-reward ratio	mean positive scenario return / $\text{abs}(\text{mean negative scenario return})$.	Rewards asymmetric payoff quality.

10.1 Kelly should be fractional and bounded

Kelly sizing maximizes log-growth under strong assumptions, but it is extremely sensitive to estimation error. In a journal-quality pipeline it should be a post-selection multiplier, not a dictator of final weights. A robust rule is to compute a Kelly-like score from scenarios, then clip it and combine it with risk-reward, signal quality, and volatility scaling. The optimizer still controls diversification and constraints.

```
base_weight = optimizer_weight
quality_multiplier = clipped_function(signal_quality)
kelly_multiplier = 0.5 + 0.5 * clipped(kelly_fraction / kelly_cap)
rr_multiplier = clipped(log(1 + RR) / log(1 + RR_cap))
vol_multiplier = target_vol / forecast_vol, clipped to [min_overlay, 1]
final_weight_raw = base_weight * quality_multiplier * kelly_multiplier * rr_multiplier * vol_multiplier
final_weight = normalize_and_cap(final_weight_raw)
```

11. Train/test validation and extended metrics

A major weakness of many portfolio backtests is full-sample parameter selection. The code now supports train/test reporting and additional metrics. In the manuscript, the selection protocol must be explicit: parameters are selected on training or validation windows, then evaluated on later unseen windows. If there are multiple variants, the reported Sharpe should be deflated over the real number of strategy variants considered.

Metric	Definition	Why it matters
Win rate	fraction of positive monthly returns.	Describes hit rate but not payoff magnitude.
Average drawdown	mean underwater drawdown when below peak.	Captures persistent pain, not only the worst point.

Expectancy	$\text{win_rate} * \text{avg_win} + (1 - \text{win_rate}) * \text{avg_loss}$.	Average expected return per rebalance month.
Profit factor	gross positive returns / absolute gross negative returns.	Useful payoff asymmetry summary.
Rolling Sharpe	Sharpe ratio over moving 6, 12, 24 month windows.	Stability of risk-adjusted performance.
Rolling Sortino	Downside-adjusted rolling performance.	Stability of downside protection.
Train/test metrics	same metrics computed separately before and after split date.	Detects parameter overfitting and instability.
PSR/DSR	probabilistic and deflated Sharpe diagnostics.	Controls non-normality, sample length, and multiple testing.

The paper should also include a walk-forward grid: train on 2019-2021, validate on 2022, test on 2023; then roll forward. A single split is a useful diagnostic, but a rolling walk-forward protocol is more convincing for OPRE.

12. Plotly HTML dashboard design

The dashboard is designed as a one-file research audit report that can be opened locally. It uses Plotly so lines can be hidden or shown from the legend, heatmaps are interactive, and ranges can be selected. The report is not just a cosmetic output; it is a reproducibility layer that helps verify each rebalance.

Dashboard component	Files used	Purpose
Performance and drawdown	pnl.csv, real.csv	Compare strategy growth and drawdown to SPY or equal-weight benchmark.
Metrics table	pnl.csv	Overall, train, and test performance metrics.
Rolling ratios	pnl.csv	Rolling Sharpe and Sortino stability.
Return histograms	pnl.csv	Distribution of monthly returns with selectable windows.
Turnover plot	weights.xlsx	Turnover at each rebalance date.
Eligible vs selected heatmap	weights.xlsx	Shows which ETFs are present and selected over time.
Covariance/correlation heatmap	risk_matrices.json, real.csv fallback	Shows the actual matrix exported by the model when available.
Forward risk signals	forecast_risk.csv	Visualizes model-CVaR and overlay diagnostics.
Dynamic controller	dynamic_parameter_history.csv	Shows alpha, lambda, and regime score paths.
Monthly weight/return tables	weights.xlsx, real.csv	Sortable and min/max weight-filterable rebalance tables.

For a website closer to cryptoliquiditymetrics.com, a Dash app would provide more powerful table filtering and multiple pages. However, a self-contained Plotly HTML file is more portable and easier to submit as a reproducibility artifact.

13. Known ideas versus defensible novelty

The article must be conservative about novelty. Many pieces are known: Markowitz optimization, Ledoit-Wolf covariance shrinkage, CVaR, Rockafellar-Uryasev LP, volatility-managed portfolios, expected-shortfall forecasting, 1/N as a strong benchmark, PSR/DSR, and bootstrap inference. A good paper does not pretend those are new. It shows that a specific combination solves a specific dynamic decision problem better than obvious alternatives.

Component	Known or new?	Safe wording for article
CVaR optimization	Known.	We use CVaR as a coherent risk constraint and scenario-based budget, not as a claimed invention.
Ledoit-Wolf shrinkage	Known.	We use shrinkage to stabilize high-

		dimensional ETF covariance estimates.
Realized-volatility timing	Known.	We include it as the canonical backward risk timer and comparator.
Forward options-free tail-risk forecasting	Known.	We do not claim forward tail forecasts are new.
Posterior-predictive CVaR as an exposure timer from the same scenario fan	Defensible contribution.	Model-consistent forward timer derived from the allocation engine itself.
Combining forward CVaR timer and backward realized-vol timer as product exposure policy	Defensible contribution if supported empirically.	Risk-signal diversification: two timers with different provenance warn in different states.
ETF universe filter	Operational contribution, not standalone novelty.	A causal, changing, liquidity/history-aware universe policy embedded in the full decision pipeline.
Allocation-versus-timing decomposition	Strong empirical contribution.	The edge is conditioning/risk timing rather than security selection; this is honest and publishable.

14. Mathematical propositions and proof sketches

To be OPRE-ready, the article should contain a compact theorem/proposition layer. These results do not have to be extremely deep; they must clarify why the policy is causal, feasible, convex, bounded, and economically interpretable.

Proposition 1. Causality of universe construction

If every ETF metric at rebalance t is computed only from observations dated $\leq t$ and every smoothing rule uses previous scores only through $t-1$, then the selected eligible universe A_t is measurable with respect to F_t . Proof: each metric is a deterministic function of F_t -measurable prices, volumes, and previous scores.

Proposition 2. Rank-score scale invariance

Percentile ranks are invariant under strictly increasing transformations. Therefore log volume and raw volume ordering produce the same ranking where monotonicity is preserved, and combining rank scores reduces unit-scale sensitivity.

Proposition 3. CVaR LP exactness

For a fixed scenario matrix, empirical CVaR is convex in weights and can be represented by the Rockafellar-Uryasev auxiliary variables η and u_s . With L1 turnover variables, the optimization is a linear program.

Proposition 4. Equal-weight tail-risk budget guarantee

If the equal-weight portfolio is feasible and $c_t = \text{CVaR}_\alpha(w_{\text{equal}})$, then the feasible set is nonempty and every optimizer solution has predicted CVaR no larger than the equal-weight budget.

Proposition 5. Partial rebalancing turnover bound

For $w_{\text{exec}} = w_{\text{prev}} + a(w_{\text{target}} - w_{\text{prev}})$, with a in $[0,1]$, turnover $\|w_{\text{exec}} - w_{\text{prev}}\|_1 = a \|w_{\text{target}} - w_{\text{prev}}\|_1 \leq \|w_{\text{target}} - w_{\text{prev}}\|_1$.

Proposition 6. Bounded dynamic controller

If the performance and regime scores are clipped to $[-1,1]$ and α and λ are clipped to configured intervals, then the adaptive policy cannot generate explosive optimizer parameters.

Proposition 7. Exposure scaling and CVaR homogeneity

Because CVaR is coherent and positively homogeneous, scaling the risky sleeve by $k \geq 0$ scales the risky component of CVaR by k , up to the deterministic cash translation.

Proposition 8. Product-timer complementarity

If k_{vol} and k_{cvar} are exposure fractions in $[k_{min}, 1]$, then $k_{product} = k_{vol} k_{cvar} \leq \min(k_{vol}, k_{cvar})$ when both are ≤ 1 . Therefore the product de-risks whenever either signal de-risks, giving broader warning-event coverage, at the cost of possible lower average exposure.

15. Empirical design, hypotheses, and inference

The empirical section should be organized around falsifiable research questions, not only around final charts.

Research question	Test	Expected evidence
RQ1: Signal validity	Correlate model-CVaR at rebalance t with next-month realized volatility, drawdown, and tail loss.	Positive relation with realized risk, not necessarily with return direction.
RQ2: Complementarity	Correlation between realized-vol exposure path and forward-CVaR exposure path; warning-event overlap table.	Weak correlation and non-overlapping warning months.
RQ3: Combination performance	Compare combined overlay with forward-only, backward-only, 1/N, Markowitz max-Sharpe, Markowitz min-var, and static allocator.	Best or near-best downside metrics; Sharpe evidence reported with DSR caveats.
RQ4: Source of edge	Decompose static allocator vs 1/N, dynamic base vs static, overlay vs no overlay.	Security selection alone close to 1/N; overlay explains most risk improvement.
Robustness	Alternative universe sizes, no SPY filter, cost levels, covariance estimators, overlay combination forms, train/test splits.	Result survives reasonable variations, especially drawdown reduction.

Required output tables: main performance table, downside-risk table, statistical inference table, universe diagnostics, overlay ablation table, train/test stability table, and trial-count table for DSR.

16. How to write the OPRE manuscript section by section

Manuscript section	What to write
Abstract	200 words maximum. State decision problem, method, main result, and contribution. Avoid equations.
Introduction	Start from monthly ETF portfolio decision under changing eligibility and estimation error. Explain why risk control, not return prediction, is central.
Literature review	Group literature into optimization/estimation error, CVaR/tail risk, volatility timing, forward ES/CVaR forecasting, and backtest inference.
Model and policy	Define state, universe, scenario engine, optimization problem, execution, and overlays.
Algorithm	Provide one clean monthly walk-forward pseudocode block and explain implementation details.
Theory/propositions	Include the proposition layer listed above; move longer proofs to appendix.
Data and universe	Describe ETF data, point-in-time universe construction, filters, missing data, and survivorship-bias controls.
Experiments	Define benchmarks, transaction costs, parameter selection, train/test protocol, and all model variants.
Results	Lead with mechanism: signal validity, complementarity, then performance and inference.
Robustness	Alternative universes, costs, matrix estimators, overlay forms, and ablations.
Conclusion	State that risk-signal diversification is useful for downside control; avoid overclaiming alpha.
Electronic companion	Code map, config files, extra figures, extra proofs, and reproducibility checklist.

17. Required upgrades before submission

- Run the full historical backtest on the real ETF dataset and regenerate all figures from the exact submitted code.
- Add survivorship-bias discussion and, if possible, use a point-in-time ETF master list with delisted/merged funds.
- Calibrate transaction costs from spreads or at least run cost sensitivity at 0, 5, 10, 25, and 50 bps turnover cost.
- Freeze the strategy-selection grid before final testing and record every trial for DSR.
- Add walk-forward parameter selection and report train, validation, and test results separately.
- Compare universe filters: pure liquidity, fixed ETF list, no SPY filter, top 50/100/150/220, and randomized liquid universes.
- Add covariance robustness: sample covariance, Ledoit-Wolf, EW covariance, cleaned covariance, and forecast-vol diagonal rescaling.
- Add overlay robustness: backward-only, forward-only, product, convex blend, adaptive blend, matched average exposure.
- Add failure-case analysis for months where the combined overlay underperforms.
- Prepare code README with exact commands, environment, data schema, random seeds, output file descriptions, and reproduction steps.

18. Code usage and file map

Example console commands. Adjust config paths and data roots to your machine.

```
# Generate monthly ETF universe scores
python -m scripts.dataloader.monthly_etf_filtration_2 --prices datasets/csv/Close.csv --volumes
datasets/csv/Volume.csv --business-dates datasets/business_dates.csv --benchmark SPY --top-n 100 --
score-ema-alpha 0.5

# Run full smart Bayesian pipeline
python -m scripts.runs.run -c advanced_smart_bayesian.yaml

# Run strategy-level smart pipeline
python -m scripts.runs.strategy_run -c strategy_smart.yaml

# Notebook-parity simplified run
python -m scripts.runs.final_results_simple --configs main_dyn_off.yaml main_dyn_strong.yaml

# Rebuild dashboard from a result folder
python -m scripts.results.interactive_report --result-folder results/advanced_smart_bayesian --
output-html results/advanced_smart_bayesian/interactive_report.html
```

File	Role in final architecture
scripts/dataloader/monthly_etf_filtration_2.py	Primary EMA-smoothed monthly ETF universe filter.
scripts/dataloader/monthly_etf_filtration.py	Rolling-score alternative universe filter and robustness variant.
scripts/runs/run.py	Full main runner for data, model, optimizer, execution, audit outputs, and report generation.
scripts/runs/strategy_run.py	Strategy-level runner for faster experiments and smart-signal audit.
scripts/runs/common_postprocess.py	Shared post-processing: audits, dashboard generation, risk matrix artifact export.
scripts/models/bayessian.py	Bayesian scenario engine; stores last covariance/correlation matrix for audit export.
scripts/optimizers/bayessian.py	CVaR-aware optimizer and dynamic allocation components.
scripts/optimizers/dynamic_params.py	Dynamic alpha/lambda controller based on performance and regime.
scripts/risk_controls/smart_signals.py	ER, ATR regime, liquidity, breakout, spike, pullback, net move, GARCH-style vol, Kelly, risk-reward.

scripts/results/stats.py	Static metrics, extended metrics, train/test split, rolling ratios, and plots.
scripts/results/interactive_report.py	Self-contained Plotly HTML dashboard.
reports/sample_dashboard/interactive_report.html	Smoke-tested example dashboard based on synthetic data.
docs/ ETF_Risk_Signal_Diversification_OPRE_Article_Blueprint.pdf	This PDF report embedded into the repository package.

19. References and reproducibility checklist

Core references to include in the final manuscript: Markowitz (1952), Ledoit and Wolf (2004), Artzner et al. (1999), Rockafellar and Uryasev (2000, 2001), DeMiguel et al. (2009), Moreira and Muir (2017), Cederburg et al. (2020), Bailey and Lopez de Prado (2014), Harvey et al. (2016), Taylor (2019), Bodnar et al. (2022), Rickenberg (2020), De Nicolo (2023), and related papers from the Word document.

Item	Must be documented
Raw data	Data source, point-in-time availability, survivorship-bias treatment, benchmark series, volume fields.
Config files	Every parameter used for the reported strategy and every parameter grid used during research.
Random seeds	Scenario generation seed and any bootstrap seeds.
Outputs	pnl.csv, real.csv, weights.xlsx, forecast_risk.csv, dynamic_parameter_history.csv, risk_matrices.json, train_test_metrics.csv, interactive_report.html.
Trial accounting	Number and definition of strategy variants included in DSR.
Environment	Python version, package versions, OS, and command-line instructions.
Reproduction script	One command to rebuild all main tables and figures from raw data.
Audit dashboard	One HTML artifact for visual inspection of monthly decisions.

Final editorial warning. Do not submit the current project as a performance story only. Submit it as a decision-policy paper: a causal universe, a scenario-based coherent-risk allocation, a bounded adaptive controller, and a complementary two-signal exposure policy with out-of-sample validation.